

Dynamic Programming

1. Matrix Chain Multiplication, Dynamic Programming Optimisation

```
#include<stdio.h>

#include<conio.h>

void main()
{
    int p[10],m[10][10];

    int i,j,k,n,q,len;

    clrscr();

    printf("Enter number of matrices: ");

    scanf("%d",&n);

    printf("Enter dimensions:\n");

    for(i=0;i<=n;i++)

        scanf("%d",&p[i]);

    for(i=1;i<=n;i++)

        m[i][i]=0;

    for(len=2;len<=n;len++)
    {
        for(i=1;i<=n-len+1;i++)
        {
            j=i+len-1;

            m[i][j]=9999;

            for(k=i;k<j;k++)
            {
                q=m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];

                if(q<m[i][j])

                    m[i][j]=q;
            }
        }
    }
```

```

    }
}

printf("Minimum multiplication cost = %d",m[1][n]);

getch();
}

```

Output

Enter number of matrices: 3

Enter dimensions:

10 20 30 40

Minimum multiplication cost = 18000

18. MATRIX CHAIN MULTIPLICATION (DYNAMIC PROGRAMMING OPTIMISATION)

PSEUDOCODE

Input : Number of matrices n
Array p[0..n] of dimensions
If matrix A_i has dimension $p[i-1] \times p[i]$

Output : Minimum number of multiplications

1. Read n
2. Read p[0..n]
3. for $i = 1$ to n
4. $m[i][i] = 0$
5. for len = 2 to n
6. for $i = 1$ to $n - len + 1$
7. $j = i + len - 1$
8. $m[i][j] = 9999$
9. for $k = i$ to $j - 1$
10. $q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]$
11. if $q < m[i][j]$
12. $m[i][j] = q$
13. Print $m[1][n]$ as minimum multiplication cost

HOW IT WORKS

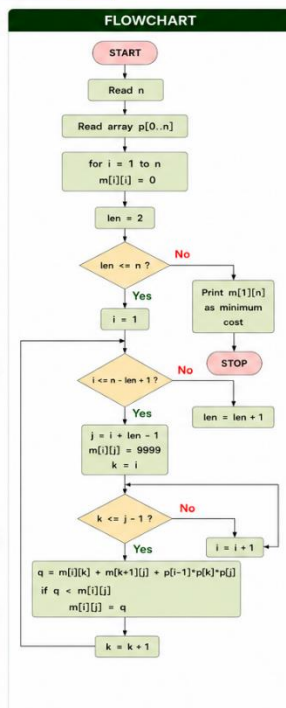
- We have a chain of matrices $A_1, A_2, \dots, A_n$.
- A_i has dimension $p[i-1] \times p[i]$.
- The goal is to find the most efficient way to multiply the chain.
- $m[i][j]$ stores the minimum cost of multiplying matrices from A_i to A_j .
- We solve the problem for chains of length 1, then length 2, and so on up to n.

COMPLEXITY

- Time Complexity : $O(n^3)$
- Space Complexity : $O(n^2)$

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of matrices: 3	Minimum multiplication cost = 18000
Enter dimensions: 10 20 30 40	



EXAMPLE					MATRICES	
Number of matrices (n) = 3					Dimensions:	
p[0]	p[1]	p[2]	p[3]	$A_1 : 10 \times 20$ $A_2 : 20 \times 30$ $A_3 : 30 \times 40$		
10	20	30	40			
DP TABLE $m[i][j]$						
i \ j	1	2	3			
1	0	6000	18000			
2		0	24000			
3			0			
$m[i][j]$ = Minimum cost of multiplying matrices from A_i to A_j						
CALCULATION STEPS						
Length 1 (i = j)						
$m[1][1] = 0, m[2][2] = 0, m[3][3] = 0$						
Length 2						
i = 1, j = 2						
k = 1: $q = m[1][1] + m[2][2] + p[0]*p[1]*p[2]$ $= 0 + 0 + 10*20*30 = 6000 \rightarrow m[1][2] = 6000$						
i = 2, j = 3						
k = 2: $q = m[2][2] + m[3][3] + p[1]*p[2]*p[3]$ $= 0 + 0 + 20*30*40 = 24000 \rightarrow m[2][3] = 24000$						
Length 3						
i = 1, j = 3						
k = 1: $q = m[1][1] + m[2][3] + p[0]*p[1]*p[3]$ $= 0 + 24000 + 10*20*40 = 32000$						
k = 2: $q = m[1][2] + m[3][3] + p[0]*p[2]*p[3]$ $= 6000 + 0 + 10*30*40 = 18000 \rightarrow m[1][3] = 18000$						
Minimum multiplication cost = $m[1][3] = 18000$						
SAMPLE RUN						
Input			Output			
Enter number of matrices: 3			Minimum multiplication cost = 18000			
Enter dimensions: 10 20 30 40						

2. Travelling Salesman Problem, Branch and Bound

```
#include<stdio.h>
#include<conio.h>

int a[10][10], visited[10], n, cost = 0;
void tsp(int city)
{
    int i, next;
    int min = 999;
    visited[city] = 1;
    printf("%d -> ", city + 1);
    for(i=0; i<n; i++)
    {
        if(a[city][i] != 0 && visited[i] == 0)
        {
            if(a[city][i] < min)
            {
                min = a[city][i];
                next = i;
            }
        }
    }

    if(min == 999)
    {
        printf("1");
        cost += a[city][0];
        return;
    }
    cost += min;
    tsp(next);
}

void main()
{
    int i, j;
    clrscr();
    printf("Enter number of cities: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
```

```
for(i=0; i<n; i++)
{
    for(j=0; j<n; j++)
    {
        scanf("%d", &a[i][j]);
    }
}
for(i=0; i<n; i++)
    visited[i] = 0;
printf("Path: ");
tsp(0);
printf("\nMinimum Cost = %d", cost);
getch();
}
```

Output

Enter number of cities: 4

Enter cost matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Path: 1 -> 2 -> 4 -> 3 -> 1

Minimum Cost = 80

22. TRAVELLING SALESMAN PROBLEM (BRANCH AND BOUND)

PSEUDOCODE

```
Global : a[10][10], visited[10], n, cost = 0
Procedure tsp(city)
1. visited[city] = 1
2. print city + 1
3. min = 999
4. for i = 0 to n - 1
5.   if a[city][i] != 0 and visited[i] == 0
6.     if a[city][i] < min
7.       min = a[city][i]
8.       next = i
9. if min == 999
10.  print 1
11.  cost = cost + a[city][0]
12.  return
13. cost = cost + min
14. tsp(next)
Main
1. Read n
2. Read cost matrix a[n][n]
3. for i = 0 to n - 1
4.   visited[i] = 0
5. Print "Path: "
6. Call tsp(0)
7. Print "Minimum Cost = " cost
```

HOW IT WORKS

- Start from city 1.
- From the current city, go to the nearest unvisited city.
- Repeat until all cities are visited.
- Finally, return to the starting city (1).
- The total cost is the sum of all selected edges.

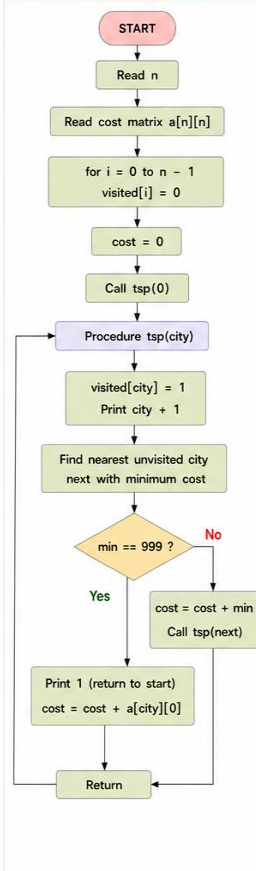
COMPLEXITY

- Time Complexity : $O(n^2)$
- Space Complexity : $O(n)$

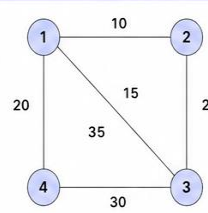
SAMPLE INPUT / OUTPUT

Input	Output
Enter number of cities: 4	Path:
Enter cost matrix:	1 -> 2 -> 4 -> 3 -> 1
0 10 15 20	Minimum Cost = 80
10 0 35 25	
15 35 0 30	
20 25 30 0	

FLOWCHART



CITIES (n = 4)



COST MATRIX a[i][j]

i \ j	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0

STEP BY STEP EXECUTION (Nearest Unvisited City)

Step	Current City	Unvisited Cities	Chosen Next City	Cost Added	Total Cost	Path So Far
1	1	{2, 3, 4}	2 (cost 10)	10	10	1 -> 2
2	2	{3, 4}	4 (cost 25)	25	35	1 -> 2 -> 4
3	4	{3}	3 (cost 30)	30	65	1 -> 2 -> 4 -> 3
4	3	{ } (None)	Return to 1 (cost 15)	15	80	1 -> 2 -> 4 -> 3 -> 1

EXPLANATION

- From city 1, nearest unvisited city is 2 (cost 10).
- From city 2, nearest unvisited city is 4 (cost 25).
- From city 4, nearest unvisited city is 3 (cost 30).
- All cities visited. Return to city 1 from city 3 (cost 15).
- Total minimum cost = 10 + 25 + 30 + 15 = 80.

SAMPLE RUN

Input (as per program)	Output (as per program)
Enter number of cities: 4	Path: 1 -> 2 -> 4 -> 3 -> 1
Enter cost matrix:	Minimum Cost = 80
0 10 15 20	
10 0 35 25	
15 35 0 30	
20 25 30 0	

3. Longest Common Subsequence (LCS), Dynamic Programming

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char x[20], y[20];
```

```
    int m, n, i, j;
```

```
    int c[20][20];
```

```
    clrscr();
```

```
    printf("Enter first string: ");
```

```
    scanf("%s", x);
```

```
    printf("Enter second string: ");
```

```
    scanf("%s", y);
```

```
    m = strlen(x);
```

```

n = strlen(y);
for(i=0; i<=m; i++)
{
    for(j=0; j<=n; j++)
    {
        if(i==0 || j==0)
            c[i][j]=0;
        else if(x[i-1]==y[j-1])
            c[i][j]=c[i-1][j-1]+1;
        else
        {
            if(c[i-1][j] > c[i][j-1])
                c[i][j]=c[i-1][j];
            else
                c[i][j]=c[i][j-1];
        }
    }
}
printf("Length of LCS = %d", c[m][n]);
getch();
}

```

Output

Enter first string: ABCDGH
Enter second string: AEDFHR
Length of LCS = 3

3. LONGEST COMMON SUBSEQUENCE (LCS) | DYNAMIC PROGRAMMING

PSEUDOCODE

Input : Two strings X and Y

Output : Length of Longest Common Subsequence

LCS(X, Y, m, n)

```
1. for i = 0 to m
2.   for j = 0 to n
3.     if i == 0 or j == 0
4.       c[i][j] = 0
5.     else if X[i-1] == Y[j-1]
6.       c[i][j] = c[i-1][j-1] + 1
7.     else
8.       if c[i-1][j] > c[i][j-1]
9.         c[i][j] = c[i-1][j]
10.      else
11.        c[i][j] = c[i][j-1]
12. return c[m][n]
```

(m = length of X, n = length of Y)

HOW IT WORKS

- Create a table $c[0..m][0..n]$.
- Rows represent string X, columns represent string Y.
- If characters match, add 1 to diagonal value.
- Else, take the maximum from top or left cell.
- The value in $c[m][n]$ is the length of LCS.

COMPLEXITY

- Time Complexity : $O(mn)$
- Space Complexity : $O(mn)$
- Works for all types of strings.

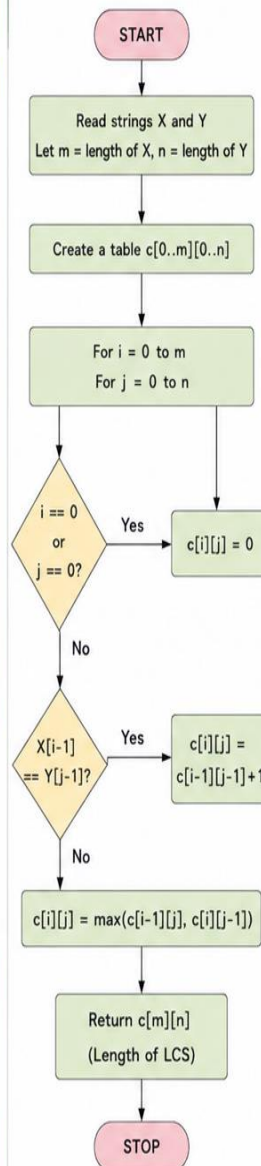
SAMPLE INPUT

Enter first string: ABCDGH

Enter second string: AEDFHR

Length of LCS = 3

FLOWCHART



EXAMPLE EXECUTION

X = A B C D G H (m = 6)

Y = A E D F H R (n = 6)

j →		0	A	E	D	F	H	R
		-	0	2	3	4	5	6
0	-	0	0	0	0	0	0	0
1	A	0	1	1	1	1	1	1
2	B	0	1	1	1	1	1	1
3	C	0	1	1	1	1	1	1
4	D	0	1	1	2	2	2	2
5	G	0	1	1	2	2	2	2
6	H	0	1	1	2	2	3	3

DRY RUN (DETAILS)

Case	Condition	Action	Example (X vs Y)
Match	$X[i-1] == Y[j-1]$	$c[i][j] = c[i-1][j-1] + 1$	$X[1]=A, Y[1]=A$ $c[1][1] = c[0][0] + 1 = 1$
No Match (Take Top)	$c[i-1][j] > c[i][j-1]$	$c[i][j] = c[i-1][j]$	Top value is greater
No Match (Take Left)	$c[i][j-1] \geq c[i-1][j]$	$c[i][j] = c[i][j-1]$	Left value is greater or equal

FINAL RESULT

Length of LCS = 3

LCS can be one of: A D H

4. 0/1 Knapsack

```
#include<stdio.h>

#include<conio.h>

void main()

{
    int i, j, n, m;

    int w[10], p[10], k[10][10];

    clrscr();

    printf("Enter number of items: ");

    scanf("%d", &n);

    printf("Enter profits:\n");

    for(i=1; i<=n; i++)

        scanf("%d", &p[i]);

    printf("Enter weights:\n");

    for(i=1; i<=n; i++)

        scanf("%d", &w[i]);

    printf("Enter capacity: ");

    scanf("%d", &m);

    for(i=0; i<=n; i++)

    {

        for(j=0; j<=m; j++)

        {

            if(i==0 || j==0)

                k[i][j] = 0;

            else if(w[i] <= j)

            {

                if(p[i] + k[i-1][j-w[i]] > k[i-1][j])

                    k[i][j] = p[i] + k[i-1][j-w[i]];

            }

        }

    }

}
```



```
        else
            k[i][j] = k[i-1][j];
    }
    else
        k[i][j] = k[i-1][j];
    }
}

printf("Maximum Profit = %d", k[n][m]);
getch();
}
```

Output: Enter number of items: 3

Enter profits:

60 100 120

Enter weights:

10 20 30

Enter capacity: 50

Maximum Profit = 220

14. 0/1 KNAPSACK

PSEUDOCODE

Input : n = number of items
 $p[1..n]$ = profits of items
 $w[1..n]$ = weights of items
 m = capacity of knapsack

Output : Maximum profit

0-1-KNAPSACK(p, w, n, m)

1. Create a table $k[0..n][0..m]$
2. for $i = 0$ to n do
3. for $j = 0$ to m do
4. if $i = 0$ or $j = 0$ then
5. $k[i][j] = 0$
6. else if $w[i] \leq j$ then
7. if $p[i] + k[i-1][j-w[i]] > k[i-1][j]$ then
8. $k[i][j] = p[i] + k[i-1][j-w[i]]$
9. else
10. $k[i][j] = k[i-1][j]$
11. else
12. $k[i][j] = k[i-1][j]$
13. return $k[n][m]$

HOW IT WORKS

- Build a table $k[i][j]$ where $i = 0..n$ and $j = 0..m$.
- $k[i][j]$ stores the maximum profit using first i items with capacity j .
- If the current item's weight is greater than j , copy the value from above $k[i-1][j]$.
- Otherwise, choose the maximum between (include item) or (exclude item).
- The answer is stored in $k[n][m]$.

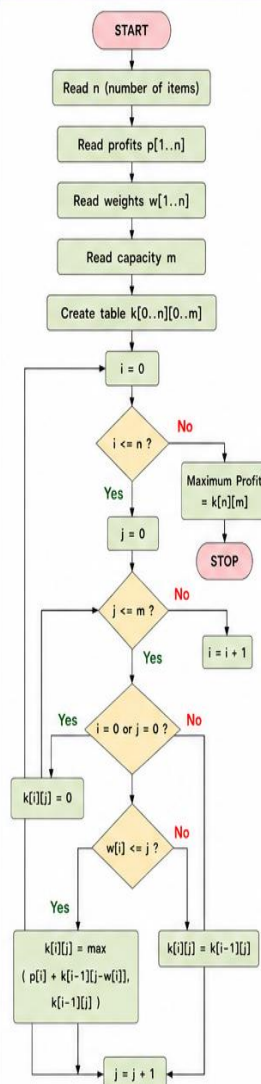
COMPLEXITY

- Time Complexity : $O(n \times m)$
- Space Complexity : $O(n \times m)$

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of items: 3	
Enter profits: 60 100 120	
Enter weights: 10 20 30	
Enter capacity: 50	Maximum Profit = 220

FLOWCHART



ITEMS

Item (i)	Profit p[i]	Weight w[i]	Profit/Weight Ratio
1	60	10	6.00
2	100	20	5.00
3	120	30	4.00

INPUT

Number of items (n): 3
 Profits: 60 100 120
 Weights: 10 20 30
 Capacity (m): 50

DP TABLE $k[i][j]$ (Maximum Profit using first i items with capacity j)

i \ j	0	10	20	30	40	50
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	60	100	160	160	160
3	0	60	100	120	180	220

Rows: $i = 0..n$ (items considered) Columns: $j = 0..m$ (capacity)

HOW THE TABLE IS FILLED (Example Explanations)

i	j (Capacity)	Decision	Calculation	$k[i][j]$
1	10	$w[1]=10 \leq 10$	$\max(p[1]+k[0][0], k[0][10]) = \max(60, 0)$	60
1	20	$w[1]=10 \leq 20$	$\max(p[1]+k[0][10], k[0][20]) = \max(60, 0)$	60
2	20	$w[2]=20 \leq 20$	$\max(p[2]+k[1][0], k[1][20]) = \max(100, 60)$	100
2	30	$w[2]=20 \leq 30$	$\max(p[2]+k[1][10], k[1][30]) = \max(160, 60)$	160
3	50	$w[3]=30 \leq 50$	$\max(p[3]+k[2][20], k[2][50]) = \max(220, 160)$	220

SELECTED ITEMS (Backtracking)

Start at $k[3][50] = 220$

- $k[3][50] \neq k[2][50] \Rightarrow$ Item 3 is included ($w=30$)
 New capacity = $50 - 30 = 20$
- $k[2][20] \neq k[1][20] \Rightarrow$ Item 2 is included ($w=20$)
 New capacity = $20 - 20 = 0$
- Capacity = 0 $\Rightarrow$ Stop

Items included in optimal solution

Item	Profit	Weight
2	100	20
3	120	30
Total Profit = 220		

Maximum Profit = 220